

SWIPEX

Smart Job Matching and Recruitment Platform

PROJECT IMPLEMENTATION REPORT

Infosys Springboard Internship 7.0 – Virtual Internship

This report documents the development, implementation, integration, testing and deployment-oriented architecture of SwipeX, including job seeker and recruiter workflows, resume analysis, recommendations, swipe-based discovery, applications, notifications, analytics and Supabase PostgreSQL storage.

Prepared by
Perumal Hema

PROJECT INFORMATION

Project: SwipeX – Smart Job Matching and Recruitment Platform

Program: Infosys Springboard Internship 7.0 – Virtual Internship

Project Type: Full-stack web application

Primary Users: Job Seekers and Recruiters

Database: PostgreSQL through Supabase

Development Environment: VS Code, Git/GitHub and API testing tools

DOCUMENT PURPOSE

This report focuses on what was developed and how the application was implemented. It describes the application flow, frontend and backend responsibilities, database persistence, resume and recommendation workflows, recruiter and job seeker functionality, smart alerts, testing considerations and deployment architecture.

The report intentionally does not include a college name. It is prepared specifically in the context of the Infosys Springboard Internship 7.0 virtual internship.

TABLE OF CONTENTS

1. Project Overview
 2. Problem Statement
 3. Project Objectives
 4. Overall Architecture
 5. Development and Integration Approach
 6. Authentication and Role-Based Access
 7. Job Seeker Implementation
 8. Recruiter Implementation
 9. Job Management and Discovery
 10. Resume Analysis Implementation
 11. Recommendation Engine
 12. Swipe, Saved Jobs and Applications
 13. Supabase PostgreSQL Database
 14. Smart Notifications and Alerts
 15. Analytics and Application Tracking
 16. Testing, Data Isolation and Security
 17. Deployment Architecture
 18. Results and Future Enhancements
 19. Conclusion
- References

1. Project Overview

SwipeX is a smart job matching and recruitment platform designed to connect job seekers with relevant job opportunities and provide recruiters with tools for job publishing and application management. The platform uses two different user experiences so that recruiters and job seekers can access features appropriate to their roles.

The job seeker side focuses on discovering jobs, searching and filtering, resume upload, personalized recommendations, swipe cards, saved jobs, applications, notifications, profile management and activity analytics. The recruiter side focuses on adding jobs, managing posted opportunities, reviewing applications, viewing available candidate resumes and monitoring recruitment-related information.

The application is implemented as a full-stack system in which the frontend communicates with backend APIs and persistent data is stored in PostgreSQL through Supabase.

- Role-specific dashboards
- Interactive job discovery
- Resume-driven matching
- Swipe-based interaction
- Application management
- Smart alerts and analytics

2. Problem Statement

Job seekers can find it difficult to identify suitable opportunities when many jobs are available. Manually checking every posting takes time and does not always provide personalized results. Recruiters, meanwhile, need a way to publish job requirements and review candidate applications efficiently.

SwipeX addresses this problem by combining conventional search and filtering with resume-based recommendations and an interactive swipe interface. It also maintains user activity and application records so that the system can provide more relevant information and prevent one user's private activity from appearing in another user's account.

3. Project Objectives

The main objective is to develop an integrated platform that supports the complete journey from job creation and discovery to recommendation, saving, applying and application tracking.

- Provide separate recruiter and job seeker experiences.
- Allow recruiters to create and manage job postings.
- Allow job seekers to search and filter jobs.
- Enable resume upload and skill-based analysis.
- Generate personalized job recommendations.
- Provide swipe-based job discovery.
- Persist saved jobs, swipes and applications.
- Provide application status tracking.
- Generate useful notifications and opportunity alerts.
- Present activity and recommendation information through dashboards and charts.

4. Overall Architecture

SwipeX uses a layered architecture. The frontend provides the user interface and communicates with backend APIs. The backend performs authentication, validation, business logic and database

operations. Supabase provides PostgreSQL persistence and can provide additional services such as authentication and storage where configured.

The logical request flow is: user → frontend → backend API → Supabase PostgreSQL → backend response → frontend display. Protected operations use the authenticated user's identity so that records such as saved jobs, swipe history, applications and resumes belong to the correct user.

- Presentation layer – job seeker and recruiter interfaces.
- API/business layer – authentication, jobs, applications, resumes, recommendations, swipes and notifications.
- Data layer – Supabase PostgreSQL.
- File layer – resume storage where configured.
- Security layer – authentication, authorization, environment variables and database access controls.

5. Development and Integration Approach

The application was developed as connected frontend and backend modules. Frontend screens collect user input and display results. Backend endpoints validate requests and perform operations against persistent data. Database records are then returned to the appropriate interface.

During integration, workflows must be checked end-to-end rather than only at screen level. For example, when a recruiter creates a job, the record must be stored in the database and then become available to job seekers. Similarly, when a job seeker swipes right, the swipe and saved-job records must be persisted and visible in the saved list.

- Design user interfaces for both roles.
- Implement API endpoints for required operations.
- Connect API requests from the frontend.
- Store and retrieve records from Supabase PostgreSQL.
- Test complete user workflows.
- Replace demonstration/fake dashboard values with database-derived values.

6. Authentication and Role-Based Access

SwipeX requires authenticated users to receive the correct application experience. The role associated with an authenticated account determines whether the user receives recruiter functionality or job seeker functionality.

Protected operations should be authorized on the backend and, where Supabase Row Level Security is used, protected at database level. A user identifier should come from the authenticated context rather than being trusted from arbitrary client input.

- Registration and login.
- Authenticated session handling.
- Recruiter-specific access.
- Job seeker-specific access.
- Protected profile and application data.
- User-specific history and dashboard records.

7. Job Seeker Implementation

The job seeker home page presents available jobs from companies. Selecting a job should open the job detail page, where the available job information such as title, company, skills, description, salary, location and job type can be displayed.

The Discover Jobs section supports searching and filtering based on criteria such as salary, job type, skills and location. A job seeker can inspect a relevant opportunity and use save/apply actions.

The resume section allows the user to upload a resume. The extracted skills become inputs to recommendation and matching workflows. The swipe section presents job cards and records the user's interaction.

The dashboard is intended to display the authenticated user's own activity, such as discovered jobs, swipes, saved jobs, applications and recommendations. The profile area provides access to editable personal information.

- Home page and job cards
- Search and filters
- Job details
- Resume upload
- Recommendations
- Swipe cards
- Saved jobs
- Applications
- Notifications
- Profile editing
- Personal activity dashboard

8. Recruiter Implementation

The recruiter dashboard is designed around job creation and hiring activity. The recruiter can add a job by entering job title, company, job type, required skills, salary, location and description.

After submission, the job record is stored in Supabase PostgreSQL. The job can then be returned by job discovery APIs so that job seekers can see the recruiter-created opportunity.

Recruiters can view applications for their jobs. When an applicant has uploaded a resume and the implemented permissions allow recruiter access, the recruiter can view the resume. If no resume exists, the application interface should indicate that the resume is unavailable rather than displaying fabricated data.

Recruiter analytics should be calculated from actual database records, including application counts, application statuses, views and relevant candidate information where those metrics are implemented.

- Add job
- Manage posted jobs
- View applications
- Candidate resume access
- Application status tracking
- Recruiter analytics

9. Job Management and Discovery

Job management is the connection between recruiter activity and job seeker discovery. A recruiter-created job must have a persistent database record and be returned by the job listing APIs. This ensures that the swipe cards and search results show actual jobs created through the application.

Job discovery uses search and filter criteria to reduce the list of opportunities. Job details provide the complete context required before saving or applying.

- Recruiter creates job → database record created.
- Job listing API retrieves active jobs.
- Job seeker searches/filters jobs.
- Job detail page displays job information.
- Job can be saved or applied for.
- User interaction is recorded for future activity and recommendations.

10. Resume Analysis Implementation

The resume analysis workflow starts with resume upload by the job seeker. The uploaded content is processed to identify relevant skills. These skills can be normalized and stored against the authenticated user.

The resulting skill profile is then used by the recommendation engine to compare a user's capabilities with job requirements. The exact parsing library, extraction rules and score calculations should correspond to the implementation used in the project.

- Upload resume.
- Extract readable content.
- Identify relevant skills.
- Normalize skill names.
- Store skills for the correct user.
- Use skills during job matching.

11. Recommendation Engine

The SwipeX recommendation approach is based on multiple signals. Resume skills are compared with required job skills, while experience and user preferences can provide additional relevance. User behavior such as right swipes, saved jobs and applications can also act as positive signals. Left-swiped jobs can be excluded from repeated recommendations.

A conceptual weighted recommendation model can combine these signals. The final weights must reflect the implemented code rather than being treated as a universal formula. For a high-match opportunity alert, a configured threshold such as 85% can be used if that threshold is actually present in the implementation.

Recommendation insights can also use job activity such as recent posting or interaction volume to identify trending opportunities. Low-competition alerts should be based on actual application/competition records.

- Resume skills vs. required skills
- Experience relevance
- Salary/job-type/location preferences
- Positive swipes
- Saved jobs
- Application history
- Previously rejected jobs
- Recent job activity

12. Swipe, Saved Jobs and Applications

The swipe module provides a quick interaction model for job discovery. A left swipe represents rejection for recommendation purposes, while a right swipe represents interest.

To make the feature persistent, the right swipe should create or update a saved-job record associated with the authenticated user and job. The saved list should retrieve only that user's records. Applying from a saved job should create an application record linked to the correct job and user.

Application tracking maintains the current state of an application. Typical states include Applied, Under Review, Shortlisted, Rejected and Hired, according to the implemented application workflow.

- Swipe left → record negative interaction.
- Swipe right → save job.
- Saved jobs → review opportunity.
- Apply → create application record.
- Application → status tracking.
- User sees only their own application history.

13. Supabase PostgreSQL Database

Supabase is used as the managed PostgreSQL data platform for SwipeX. The database provides persistent storage so that actions performed in the application remain available after a page refresh or new login session.

The database should model relationships between users, recruiters, job seekers, jobs and interaction records. A recruiter can create multiple jobs, a job can receive multiple applications, and a job seeker can have multiple swipes and saved jobs.

User isolation is particularly important. Every private activity record should be linked to the authenticated user's identifier, and database policies should prevent unauthorized access.

- Users / profiles
- Jobs
- Applications
- Saved jobs
- Swipe history
- Resumes
- Resume skills
- Recommendations
- Job views/activity
- Notifications

Example relationship flow: Recruiter → Jobs → Applications ← Job Seeker. Job Seeker → Resume/Skills, Saved Jobs and Swipe History. Jobs and user activity feed the recommendation and analytics workflows.

14. Smart Notifications and Alerts

The smart communication layer is designed to make the application proactive. Instead of requiring users to repeatedly search, relevant events can generate notifications.

Instant job notifications can be triggered when a new job is added and matches the user's profile. Startup hiring alerts can inform users when a relevant company begins hiring. Personalized recommendation alerts can be generated from resume skills, experience and preferences.

High-match opportunity alerts can identify jobs above the configured matching threshold. Low-competition alerts can be generated when a relevant job has relatively low competition based on actual application data. Notifications should be generated from real events and stored or delivered through the implemented notification mechanism.

- Instant job notification
- Startup hiring alert
- Personalized skill-based alert
- High-match opportunity alert
- Low-competition relevant-job alert
- Application status notification
- Trending-job insight

15. Analytics and Application Tracking

Job seeker analytics summarize personal activity. Useful measures include jobs discovered, swipe activity, saved jobs, applications and recommendation activity. Recruiter analytics summarize recruitment activity such as applications, status distribution and job performance.

Recommendation insight tracking can show trending jobs using graphical representations such as bar charts, pie charts or other dashboard visualizations. Resume performance tracking can provide information about how effectively the resume matches available opportunities, depending on the implemented scoring model.

All dashboard values should be derived from real database records. Hardcoded demonstration numbers should not be used in the final implementation.

- Job seeker: discovered jobs, swipes, saves and applications.
- Job seeker: accepted/shortlisted/rejected outcomes where available.
- Recruiter: total applications and status distribution.
- Recruiter: job views and candidate relevance where implemented.
- Trending jobs and recommendation insights.
- Graphical and chart-based presentation.

16. Testing, Data Isolation and Security

Testing should validate both functionality and correctness of stored data. A particularly important test is to create two separate job seeker accounts and verify that activity created by one account does not appear in the other account.

Authentication and authorization must be checked for protected endpoints. Database access policies should support the same ownership rules as the backend. Secrets such as database passwords and service-role keys must be stored in environment variables and never committed to the source repository.

- Login and role testing
- Recruiter job creation persistence
- Job seeker search/filter testing
- Right swipe and saved-job persistence
- Left swipe exclusion
- Application creation and removal/status workflow
- Resume upload and recruiter access
- Notification event testing
- Dashboard calculation testing
- Two-user data-isolation testing
- RLS/authorization testing

17. Deployment Architecture

The production architecture separates the web frontend, backend API and Supabase database. The frontend can be deployed to a frontend hosting service such as Vercel, while the backend can be deployed to a service such as Render. Supabase hosts the PostgreSQL database.

After deployment, the frontend must use the public backend API URL rather than localhost or a private network address. Environment variables are used to configure API URLs, database connections and other secrets.

- GitHub → source repository
- Frontend → production hosting
- Frontend → HTTPS backend API
- Backend → Supabase PostgreSQL
- Environment variables → protected configuration
- End-to-end testing → live application

18. Results and Future Enhancements

SwipeX provides a connected recruitment workflow in which recruiters can create opportunities and job seekers can discover, evaluate, save and apply for them. Resume information and activity signals support personalized discovery, while application tracking and analytics provide visibility into recruitment activity.

Future enhancements can improve resume parsing, recommendation ranking, real-time notification delivery, trend analysis, recruiter candidate comparison, monitoring and automated testing.

- Advanced resume parsing
- Improved recommendation ranking
- Real-time event notifications
- More detailed recruiter analytics
- Advanced job-trend analysis
- Scalable monitoring and logging
- Automated integration tests

19. Conclusion

SwipeX demonstrates the implementation of a smart recruitment platform that combines job management, job discovery, resume analysis, personalized recommendations, swipe-based interactions, saved jobs, applications, notifications and analytics.

The use of Supabase PostgreSQL provides the persistent foundation for the platform. Correct authentication, authorization and user-specific database filtering are essential to protect private activity. The overall architecture provides a practical foundation for continued development and deployment as part of the Infosys Springboard Internship 7.0 virtual internship.

REFERENCES

1. Supabase documentation – PostgreSQL database, authentication, storage and Row Level Security concepts.
2. React documentation – component-based frontend application development.
3. PostgreSQL documentation – relational database, SQL and data management concepts.
4. GitHub documentation – source control and repository management.